A Post-Quantum Take on a Classical Problem

Mikhail Suslov
https://github.com/AdamaSoftware/InverseDiscrete/

July 31, 2025

Abstract

We introduce the *Inverse Discrete Logarithm Problem* (iDLP) framework, which inverts traditional discrete logarithm assumptions by keeping the *exponent public but non-invertible modulo the group order*, while hiding the base. This yields a many-to-one algebraic mapping that is irreversible under both classical and quantum models.

Within this framework, we define three post-quantum cryptographic primitives:

- **Inverse Discrete Diffie–Hellman (IDDH)**,
- **Inverse Discrete Key Encapsulation (IDKE)**,
- **Inverse Discrete Data Encapsulation (IDDE)**.

We consider a 512-bit modulus m (either prime or a product of two 256-bit primes), a random generator $g \in \mathbb{Z}_m^*$, and a public exponent y with $\gcd(y, \varphi(m)) = 2$. This setting ensures the masking function

$$\mathsf{Mask}_{g,y}(x) := g^{xy} \bmod m$$

collapses inputs into two equivalence classes, yielding a function without inverse under standard assumptions. This limits known attacks to exhaustive search, which remains infeasible at 512-bit sizes.

We demonstrate efficient implementations with sub-millisecond key operations and AES-GCM-level data throughput. Our full implementation is available at `https://github.com/AdamaSoftware/InverseDiscrete/`.

1 Introduction

Traditional discrete-logarithm cryptography relies on the secrecy and invertibility of the exponent modulo the group order. By contrast, the *Inverse Discrete Logarithm Problem* (iDLP) flips this: the exponent y is public but deliberately non-invertible modulo $\varphi(m)$, while the base g is kept secret.

Formally, given

$$c = g^{xy} \bmod m,$$

with $\gcd(y, \varphi(m)) = d > 1$ (in practice $d = 2$), recovering x is hard because the function

$$x \mapsto g^{xy} \bmod m$$

is d-to-1, collapsing the search space into equivalence classes. Even quantum algorithms such as Shor's cannot invert y and thus only yield multiple candidates, forcing exhaustive search among them.

This work formalizes iDLP, develops three cryptographic primitives (IDDH, IDKE, IDDE) based on it, analyzes their security under classical and quantum models, and provides benchmarks demonstrating practical efficiency.

2 Notation and Problem Statement

Let:

- m be a 512-bit modulus, either a prime or a product of two 256-bit primes,
- $\varphi(m)$ be Euler's totient function,
- $g \in \mathbb{Z}_m^*$ be a generator chosen uniformly at random,
- y be a public exponent with $\gcd(y, \varphi(m)) = d > 1$ (typically $d = 2$).

Define the masking function:

$$\mathsf{Mask}_{g,y}(x) := g^{xy} \bmod m.$$

Definition 1 (Inverse Discrete Logarithm Problem (iDLP))**.** *Given (m, g, y, c) where $c = \mathsf{Mask}_{g,y}(x)$, recover any valid x such that $c = g^{xy} \bmod m$.*

Lemma 1 (Collapsing Map)**.** *The function $\mathsf{Mask}_{g,y}$ is d-to-1 over $\mathbb{Z}_{\varphi(m)}$.*

Proof. Write $y = dy'$ and $\varphi(m) = d\varphi'$, where $\gcd(y', \varphi') = 1$. Then:

$$\mathsf{Mask}_{g,y}(x) = g^{xdy'} = (g^{y'})^{dx} \bmod m.$$

Because exponentiation by d partitions $\mathbb{Z}_{\varphi(m)}$ into d equivalence classes, the map is d-to-1. □

Proposition 1 (Reduction to Exhaustive Search)**.** *Any algorithm that recovers x from $c = g^{xy} \bmod m$ requires $\Omega(\varphi(m)/d)$ group operations under classical and quantum models.*

Sketch. Shor's algorithm can compute discrete logarithms modulo $\varphi(m)$, but since y is not invertible mod $\varphi(m)$, recovering x from $xy \bmod \varphi(m)$ yields d possible values. Distinguishing the correct preimage requires exhaustive search over these candidates. □

3 Cryptographic Primitives

3.1 Inverse Discrete Diffie–Hellman (IDDH)

- Public parameters: (m, g, y).
- Each party samples a secret x_A, x_B and sends $A = g^{x_A y} \bmod m$ and $B = g^{x_B y} \bmod m$.
- Shared secret:

$$S = g^{x_A x_B y} \bmod m.$$

Theorem 1 (Correctness)**.** *Both parties derive the same S.*

Proof. Commutativity of exponentiation in $\mathbb{Z}_m^*$ yields

$$A^{x_B} = (g^{x_A y})^{x_B} = g^{x_A x_B y} = g^{x_B x_A y} = (g^{x_B y})^{x_A} = B^{x_A}.$$

□

3.2 Inverse Discrete Key Encapsulation (IDKE)

- Sender picks ephemeral x_S, computes $A = g^{x_S y} \bmod m$.
- Receiver's public key is $B = g^{x_B y} \bmod m$.
- Shared secret $U = (A^{x_B})^y \bmod m$.
- Derive symmetric key $K = \mathsf{KDF}(U)$.
- Ciphertext: A.

3.3 Inverse Discrete Data Encapsulation (IDDE)

- Use IDKE to derive shared key K.
- Encrypt plaintext π with AES-GCM or any symmetric authenticated encryption keyed by K.
- Output ciphertext $(A, \text{AES-GCM}_K(\pi))$.

The KDF can be any standard key derivation function (e.g., SHA-512, Argon2, HKDF).

4 Security Model and Analysis

4.1 Security Intuition

The core security of iDLP rests on the non-invertibility of the exponent y. Since $\gcd(y, \varphi(m)) = d > 1$, standard discrete log inversion is impossible, yielding d candidate solutions.

Shor's quantum algorithm computes discrete logs but still returns $w = xy \bmod \varphi(m)$, providing no unique x. The best known attack is thus to test these d candidates exhaustively.

For $d = 2$ and 512-bit modulus, the search space is roughly 2^{511}, prohibitive for classical or foreseeable quantum computers.

4.2 Resistance to Known Attacks

- **Classical DLP**: Shor's algorithm breaks invertible exponents but yields multiple candidates here.
- **Grover's algorithm**: Gives quadratic speedup on exhaustive search, still requiring 2^{255} operations—secure for current parameters.
- **Other algebraic attacks**: No known algorithms exploit the structure for faster inversion due to the masking map's many-to-one property.

4.3 Formal Security Statements

We conjecture that the hardness of iDLP with parameters (m, g, y) where $\gcd(y, \varphi(m)) = d$ reduces to exhaustive search over d classes modulo $\varphi(m)$. We leave full reductions and formal proofs for future work.

5 Implementation and Benchmarks

We implemented IDDH, IDKE, and IDDE in Python (non-optimized). All operations use 512-bit modular arithmetic and standard KDFs/AES-GCM.

Benchmarks on typical modern hardware show:

- IDKE encapsulation-decapsulation cycle: ~ 2 ms.
- Key generation: ~ 600 µs.
- IDDE encrypt-decrypt cycle: ~ 1 second per GB, due to AES-GCM efficiency.

Parameters used in implementation satisfy the required $\gcd(y, \varphi(m)) = 2$. See our full source code and parameters at `https://github.com/AdamaSoftware/InverseDiscrete/`.

6 Comparison to Existing Post-Quantum Assumptions

- **Classical discrete log** (e.g., DSA, ECDSA): broken by quantum algorithms.
- **Lattice-based cryptography** (e.g., LWE, NTRU): quantum-secure but involves large keys and complex operations.
- **Code-based cryptography** (e.g., McEliece): secure but with very large public keys.
- **Isogeny-based cryptography** (e.g., SIKE): compact keys but slow performance.

iDLP offers a novel middle ground: algebraic hardness rooted in structural non-invertibility, with compact keys and efficient arithmetic operations, resistant to both classical and quantum attacks.

Acknowledgments

Thanks to the open cryptographic community and open-source contributors who have inspired this work.

Appendix: Concrete Parameters

Modulus m:

```
787519524829957380882371229197888827444299995967769475391257184778008721400272285868368906952
```

Euler's totient $\varphi(m)$:

```
787519524829957380882371229197888827444299995967769475391257184778008721400254535603753498927
```

Public exponent y:

```
629922990883384011861982967294575141173562260565628902551291459651817715938340005972214226904
```

Generator g:

30528800111333176445658323344859216589886910080532368058426695521042845849716583293335691362941

These parameters satisfy $\gcd(y, \varphi(m)) = 2$ and were used in all benchmarks and implementations.

Update and Correction (August 2025)

After publication, it was pointed out in Paper 2025/1417 [7] that the original security claim—stating the hardness of iDLP as approximately $\frac{2^{512}}{d}$—was overly optimistic.

The corrected security level corresponds precisely to $d + 1$ candidate equivalence classes due to the $\gcd(y, \varphi(m)) = d$ masking, not exponential in the modulus size. In our updated parameters, d is a 256-bit number, which sets the security level to approximately 256 bits. This means the exhaustive search complexity is linear in $d + 1$, reflecting a security level consistent with 2^{256} operations, rather than $2^{512}/d$.

Accordingly, we update the parameters used in our benchmarks to:

$$
\begin{aligned}
m &= 10149176142605696715050247890886518684869103063509537620648493592653337479126199564779492324 \\
\varphi(m) &= 10149176142605696715050247890886518684869103063509537620648493592653337479125996503442675300 \\
g &= 75917502382830222265916242924398568612661145909487255960899238693157455867493997367276711189 \\
y &= 28910565679645364197922634190041392732494728077485656916113706530410198298819033139809956 50
\end{aligned}
$$

where

$$d = \gcd(y, \varphi(m))$$

is a 256-bit integer.

These parameters satisfy the updated security assumptions consistent with the corrected analysis.

We thank the authors of the paper for their careful review and valuable feedback, which has strengthened the rigor of this work.

References

[1] P. W. Shor, *Algorithms for quantum computation: discrete logarithms and factoring*, in Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS), 1994.

[2] O. Regev, *On lattices, learning with errors, random linear codes, and cryptography*, Journal of the ACM, 56(6):34, 2009.

[3] J. Hoffstein, J. Pipher, J. H. Silverman, *NTRU: A ring-based public key cryptosystem*, in PQCrypto 1998.

[4] R. J. McEliece, *A public-key cryptosystem based on algebraic coding theory*, DSN Progress Report, 1978.

[5] D. Jao, L. De Feo, *Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies*, PQCrypto 2011.

[6] T. Castryck et al., *CSIDH: An efficient post-quantum commutative group action*, ASIACRYPT 2018.

[7] J. Limbrey, A. Mendelsohn, *A Note on the Post-Quantum Security of the Inverse Discrete Logarithm Problem*